

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Курсова робота

з дисципліни «Інженерія програмного забезпечення»

на тему: Планувальник завдань: розподіл та контроль за виконанням завдань
членами команди проекту

Студента 2 курсу ІО-35 групи

Спеціальності:

123 «Комп’ютерна інженерія»

Степанов О. О.

Керівник Марія ВАСИЛЬЄВА

Національна оцінка _____

Кількість балів: _____

Оцінка: ECTS _____

Члени комісії _____

(підпис) (вчене звання, науковий ступінь, прізвище та ініціали)

(підпис) (вчене звання, науковий ступінь, прізвище та ініціали)

(підпис) (вчене звання, науковий ступінь, прізвище та ініціали)

Київ – 2025 рік

Кафедра	обчислювальної техніки				
Дисципліна	Інженерія програмного забезпечення				
Спеціальність	123 “Комп’ютерна інженерія”				
Курс	<u>2</u>	Група	<u>ІО-35</u>	Семестр	<u>4</u>

ЗАВДАННЯ

НА КУРСОВУ РОБОТУ СТУДЕНТУ

Степанов Александр Александрович

1. Тема Планувальник завдань: розподіл та контроль за виконанням завдань членами команди проекту
2. Термін здачі студентом закінченого проєкту _____ травня 2025 р.
3. Завдання курсової роботи Розробити специфікацію вимог програмного забезпечення, яка містить від 10 до 20 функціональних (F) та нефункціональних (NF) вимог. Обрати та впровадити відповідні патерни проєктування у розробці програмного продукту. Спроекувати UML- діаграми для основних компонентів системи, що реалізують патерни проєктування. Реалізувати програму відповідно до сформованих вимог, дотримуючись принципів ООП, Composition over Inheritance, Loose Coupling – High Cohesion, Inversion of Control (IoC) та принципів SOLID. Розробити модульні тести для ключових компонентів бізнес-логіки, використовуючи відповідні інструменти тестування (JUnit, NUnit або інші відповідно до мови програмування).
4. Зміст розрахунково-пояснювальної записки
Анотація. Вступ. Постановка задачі.
Проектування програмного додатку
Розробка програмного додатку.
Тестування програмного додатку. Висновки.
5. Перелік графічного матеріалу _____
6. Дата видачі завдання « » лютого 2025 р.

Календарний план

	Найменування етапів виконання курсової роботи	Терміни виконання етапів роботи	Примітки
1.	<i>Отримання теми та завдання</i>	20 лютого 2025 р.	
2.	<i>Підбор та вивчення літератури</i>	21–27 лютого 2025 р.	
3.	<i>Формування технічного завдання</i>	28–29 лютого 2025 р.	
4.	<i>Огляд методів колективної розробки програмного забезпечення</i>	1–5 березня 2025 р.	
5.	<i>Розробка прецедентів розроблюваної програмного додатка</i>	6–15 березня 2025 р.	
6.	<i>Розробка програмного додатка</i>	16 березня – 30 квітня 2025 р.	
7.	<i>Тестування програмного додатку</i>	30 квітня – 7 травня 2025 р.	
8.	<i>Оформлення пояснювальної записки</i>	8–10 травня 2025 р.	
9.	<i>Подання курсового проекту (роботи) на перевірку</i>	30 травня 2025 р.	
10.	<i>Захист курсового проекту (роботи)</i>	4 червня 2025 р.	

Студент

(підпис)

Керівник

_____ *Марія ВАСИЛЬЄВА*

(підпис)

« » лютого 2025 р.

Анотація

У курсовій роботі розроблено вебзастосунок «Планувальник завдань» для організації ефективної взаємодії між учасниками проєктної команди. Система дозволяє створювати, редагувати та контролювати виконання задач у межах обраного проєкту, а також призначати відповідальних користувачів. Основна мета роботи — реалізація надійного програмного забезпечення з використанням сучасних вебтехнологій, а також проведення модульного тестування для перевірки коректності бізнес-логіки.

Проєкт реалізований із дотриманням архітектури MVC. Для клієнтської частини використано стек React, TypeScript та Vite. Серверна частина побудована на базі Node.js з використанням Express.js, Prisma ORM та PostgreSQL. Такий технологічний стек забезпечує гнучкість, масштабованість і легкість підтримки проєкту.

У ході роботи було реалізовано CRUD-операції для користувачів, задач, проєктів та призначень. Логіку взаємодії з базою даних винесено у сервісний рівень, що дало змогу зосередити тестування саме на ньому. Було написано 37 модульних тестів із використанням бібліотеки Vitest, що забезпечило повне (100%) покриття коду основних сервісів. Для перевірки поведінки системи на рівні API також проводилося функціональне тестування вручну за допомогою Postman.

Всі заплановані функції реалізовано, система показала стабільну роботу, а результати тестування підтвердили правильність реалізованої логіки. Курсову роботу можна вважати успішно виконаною, а отриманий програмний продукт — готовим до використання у малих командах або як основа для подальшого розширення.

Технічне завдання

Технічне завдання на курсову роботу

Зміст

1. НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ВИКОРИСТАННЯ	5
2. ПРИЧИНИ ДЛЯ РОЗРОБКИ	5
3. МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ.....	5
4. ДЖЕРЕЛА РОЗРОБКИ.....	5
5. ТЕХНІЧНІ ВИМОГИ	6
5.1 ВИМОГИ ДО ПРОДУКТУ, ЩО РОЗРОБЛЯЄТЬСЯ.....	6
5.2 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
5.3 ВИМОГИ ДО АПАРАТНОЇ ЧАСТИНИ	6
6. ЕТАПИ РОЗРОБКИ	7

1. НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ВИКОРИСТАННЯ

Це технічне завдання поширюється на розробку програмного засобу "Планувальник завдань", призначеного для розподілу та контролю виконання завдань між членами команди проєкту. Застосунок орієнтований на малий та середній бізнес, команди розробників, навчальні та дослідницькі групи, де необхідно забезпечити планування, моніторинг та аналіз виконання завдань у межах проєктної діяльності.

2. ПРИЧИНИ ДЛЯ РОЗРОБКИ

Підставою для розробки є завдання на виконання курсової роботи за освітньо-професійною програмою "Комп'ютерна інженерія" спеціальності 123 "Комп'ютерна інженерія".

3. МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ

Метою даного проєкту є створення вебзастосунку для централізованого управління завданнями в команді, що дозволить:

- ефективно розподіляти задачі між учасниками;
- контролювати прогрес виконання завдань;
- здійснювати пошук інформації за виконавцями та статусами завдань;
- підвищити прозорість виконання командних проєктів.

4. ДЖЕРЕЛА РОЗРОБКИ

Джерелами для розробки є науково-технічна література з проєктування програмних систем, офіційна документація технологій (React, Node.js, TypeScript, PostgreSQL), публікації у фахових періодичних виданнях, довідники з архітектурного проєктування вебзастосунків, а також матеріали Інтернет-ресурсів, що стосуються розробки програмного забезпечення.

5. ТЕХНІЧНІ ВИМОГИ

5.1 Вимоги до продукту, що розробляється

- Реалізація функціоналу управління користувачами та завданнями відповідно до специфікації.
- Забезпечення можливості пошуку та фільтрації завдань за різними критеріями.
- Підтримка відображення прогресу виконання проєкту.
- Інтуїтивно зрозумілий інтерфейс користувача.
- Можливість масштабування функціоналу в майбутньому.

5.2 Вимоги до програмного забезпечення

- Операційна система сервера: Windows або Linux.
- Середовище виконання: Node.js (версія 18 або вище).
- Фронтенд: React (версія 18 або вище).
- База даних: PostgreSQL (версія 13 або вище).
- Інструменти розробки: TypeScript, Express.js, Prisma ORM.
- Використання браузерів: Google Chrome, Mozilla Firefox, Microsoft Edge (актуальні версії).

5.3 Вимоги до апаратної частини

- Процесор: не нижче Intel Core i5 або аналогічний.
- Оперативна пам'ять: мінімум 8 ГБ.
- Дисковий простір: не менше 20 ГБ вільного місця.
- Підключення до мережі Інтернет.

6. ЕТАПИ РОЗРОБКИ

- 6.1 Отримання теми та завдання на курсову роботу – **20 лютого 2025 р.**
- 6.2 Підбір та вивчення літературних і технічних джерел – **21–27 лютого 2025 р.**
- 6.3 Формування технічного завдання – **28–29 лютого 2025 р.**
- 6.4 Огляд архітектури MVC та шаблонів проєктування – **1–5 березня 2025 р.**
- 6.5 Проєктування бази даних та розробка ER-моделі – **6–15 березня 2025 р.**
- 6.6 Розробка вебзастосунку: бекенд та фронтенд частини – **16 березня – 30 квітня 2025 р.**
- 6.7 Тестування функціональності програмного забезпечення – **30 квітня – 7 травня 2025 р.**
- 6.8 Оформлення пояснювальної записки – **8–10 травня 2025 р.**
- 6.9 Подання курсового проєкту на перевірку – **30 травня 2025 р.**
- 6.10 Захист курсової роботи – **4 червня 2025 р.**

ЗМІСТ

РОЗДІЛ 1. ТЕХНІЧНЕ ЗАВДАННЯ	10
1.1 Постановка задачі	10
1.2 Функціональні вимоги	10
1.3 Огляд архітектури MVC	11
1.3.1 Основні принципи MVC	11
1.3.2 Компоненти MVC та їх взаємодія	12
1.3.3 Схема взаємодії компонентів MVC	Error! Bookmark not defined.
1.3.4 Переваги та недоліки MVC	13
РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	15
2.1 Визначення вимог до системи	15
2.1.1 Основні сценарії використання	16
2.2 Архітектура бази даних	21
2.2.1 Основні таблиці та їх структура	22
2.2.2 Схема зв'язків між таблицями	23
2.3 Вибір технологій розробки	24
2.4 Використання шаблонів проєктування	25
2.5 Застосування принципів ООП і SOLID	27
2.6 Діаграми класів та послідовностей	Error! Bookmark not defined.
РОЗДІЛ 3. РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	29
3.1. Загальна структура коду	29
3.1.1 Frontend частина	29
3.1.2 Backend частина	30
3.2. Реалізація моделі	32
3.3. Реалізація контролерів	33
3.4. Реалізація представлення (View)	34
3.5. Логіка взаємодії між компонентами	36
3.6. Репозиторій з кодом проєкту	38
РОЗДІЛ 4. ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	39
4.1. Підходи до тестування в архітектурі MVC	39
4.2. Методи тестування	39
4.2.1. Модульне тестування	39
4.2.2. Інтеграційне тестування	41
4.2.3. Функціональне тестування	41
4.3. Аналіз результатів тестування	42
ВИСНОВКИ	43
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	44

ДОДАТКИ	45
Додаток А	46
Додаток Б	47
Додаток В	49
Додаток Г	51

РОЗДІЛ 1. ТЕХНІЧНЕ ЗАВДАННЯ.

1.1 Постановка задачі

У сучасних умовах ефективна організація командної роботи є ключовим фактором успішного виконання проєктів. При цьому важливими аспектами залишаються планування, контроль за виконанням завдань та оцінка завантаженості виконавців. Особливо це актуально в умовах розподілених команд, де комунікація ускладнена віддаленим форматом взаємодії.

Метою даної курсової роботи є розробка вебзастосунку — планувальника завдань, який дозволяє керівнику команди ефективно розподіляти задачі, контролювати їх виконання та аналізувати поточний стан проєкту.

Система має бути розроблена на основі стека технологій: TypeScript, Node.js, React та PostgreSQL, із дотриманням принципів об'єктно-орієнтованого програмування, а також патернів проєктування.

1.2 Функціональні вимоги

Програмний засіб повинен забезпечувати повноцінне управління командною роботою в межах одного або кількох проєктів. Серед ключових функціональних можливостей передбачається управління членами команди, управління завданнями, контроль їх виконання, а також пошук необхідної інформації.

Управління членами команди включає можливість додавання нових виконавців, редагування інформації про них, видалення з системи та перегляд повного списку учасників команди.

Управління завданнями передбачає створення нових задач, редагування вже наявних, їх видалення, перегляд загального списку завдань, а також фільтрацію за статусом виконання (виконано або не виконано).

Контроль за розподілом і виконанням завдань реалізується через функціонал призначення задач конкретним виконавцям, зазначення ступеня виконання, визначення статусу задачі залежно від терміну виконання (активне чи прострочене), перегляд завантаженості виконавців та оцінку загального стану виконання проєкту.

Також система повинна надавати можливість здійснювати пошук за виконавцем і його завданнями, знаходити всіх виконавців, залучених до певного завдання, а також виконувати пошук задач за статусом або термінами виконання.

1.3 Огляд архітектури MVC

Для побудови архітектури програмного забезпечення було обрано модель MVC (Model–View–Controller), яка широко застосовується у веброзробці завдяки своїй гнучкості, модульності та чіткому поділу обов'язків між складовими частинами системи. Використання шаблону MVC дозволяє ізольовано розробляти логіку обробки даних, відображення інтерфейсу користувача та взаємодію між ними, що значно спрощує супровід і розширення програмного продукту.

1.3.1 Основні принципи MVC

Архітектурний шаблон MVC передбачає поділ програмного забезпечення на три взаємопов'язані компоненти:

Model (Модель) відповідає за роботу з даними, бізнес-логіку та доступ до бази даних. У межах проєкту це сервіси й класи, які виконують валідацію, зчитування, створення та оновлення інформації в базі PostgreSQL.

View (Подання) реалізує інтерфейс користувача та відповідає за візуальне відображення даних. У даному застосунку подання побудоване за допомогою бібліотеки React і є односторінковим застосунком (SPA).

Controller (Контролер) виконує роль посередника між користувачем і логікою програми. Контролер обробляє вхідні HTTP-запити, передає дані до моделі та

повертає результати у вигляді відповіді. На сервері контролери реалізовані з використанням Express.js.

1.3.2 Компоненти MVC та їх взаємодія

У розробленому застосунку контролери обробляють HTTP-запити до REST API (наприклад, `/api/tasks`, `/api/users`) та викликають відповідні сервіси для обробки даних. Моделі реалізують основну бізнес-логіку — створення, зміну, пошук або видалення завдань, користувачів та їх зв'язків. Для доступу до бази даних використовуються репозиторії або ORM.

Клієнтська частина застосунку, реалізована на React, взаємодіє з бекендом через HTTP-запити та відображає отриману інформацію у вигляді інтерфейсу користувача. У результаті забезпечується повний цикл взаємодії між користувачем і даними через чітко структуровані компоненти архітектури.

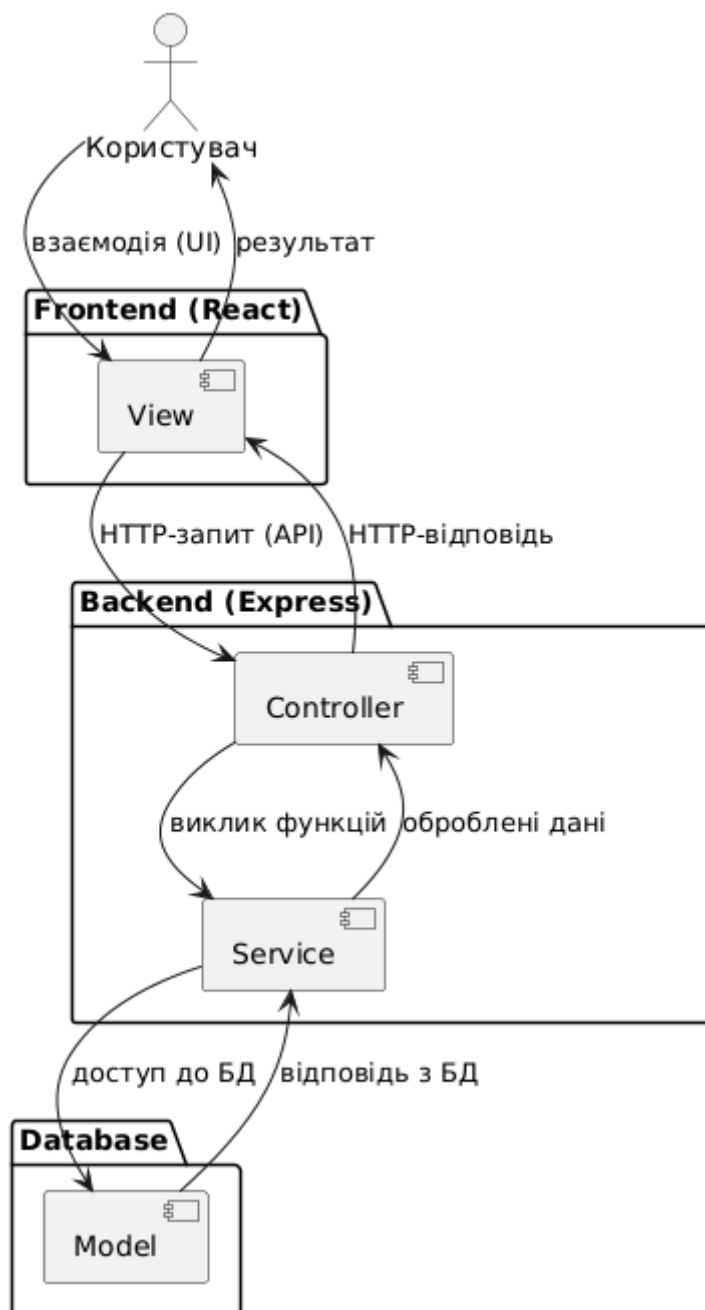


Рисунок 1.1 – Схема взаємодії компонентів шаблону MVC

1.3.3 Переваги та недоліки MVC

Використання архітектурного шаблону MVC має низку суттєвих переваг, що робить його одним із найпоширеніших підходів у проектуванні сучасних вебзастосунків. Насамперед, MVC забезпечує чітке розділення відповідальностей між компонентами: модель відповідає за логіку та дані, контролер — за обробку запитів, а подання — за взаємодію з користувачем. Такий підхід значно підвищує

читабельність та керованість коду, що полегшує тестування і подальший супровід системи.

Ще однією важливою перевагою є можливість незалежної розробки клієнтської і серверної частин, що особливо актуально в умовах командної роботи. Завдяки модульності та гнучкості архітектури MVC, її легко масштабувати та розширювати — додавання нових функцій відбувається без необхідності змінювати існуючу логіку.

Разом з тим, використання шаблону MVC має і певні недоліки. Зокрема, він вимагає чіткого планування структури застосунку ще на етапі проєктування, оскільки розподіл відповідальностей між компонентами повинен бути продуманим і послідовним. Крім того, для простих застосунків архітектура MVC може виявитися надмірно складною через багаторівневу структуру. Це створює додаткові накладні витрати під час розробки, які можуть бути недоцільними у випадках з обмеженим функціоналом.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.

2.1 Визначення вимог до системи

Метою розробки програмного забезпечення є створення вебзастосунку, який забезпечує ефективне управління завданнями всередині команди проєкту. Система має надавати можливості для адміністрування користувачів (виконавців), керування завданнями, моніторингу виконання, а також фільтрації й пошуку інформації.

До основних функціональних вимог належать:

- Додавання, редагування та видалення користувачів (виконавців);
- Додавання, редагування та видалення завдань;
- Призначення завдань конкретним виконавцям;
- Контроль статусу виконання та термінів;
- Визначення рівня завантаженості виконавців;
- Пошук за параметрами: ім'я виконавця, назва завдання, статус виконання, дедлайн.

Користувачами системи можуть бути:

- **Адміністратор / Менеджер**, який створює проєкт, додає виконавців та контролює виконання;
- **Виконавець**, який бачить призначені йому задачі, позначає їх виконання та взаємодіє із системою в межах своїх повноважень.

Система повинна забезпечити авторизацію, обмеження доступу за ролями, а також зручний та адаптивний інтерфейс для взаємодії з основним функціоналом.

2.1.1 Основні сценарії використання

Адміністратор відкриває сторінку управління командою та ініціює процес додавання нового виконавця. Після введення необхідних даних (ПІБ, email тощо), система перевіряє їхню валідність. У разі відсутності помилок створюється новий користувач і зберігається у базі даних. Якщо виникають помилки (наприклад, некоректні дані чи вже існуючий email), адміністратор отримує відповідне повідомлення. У разі успіху надсилається підтвердження про додавання.

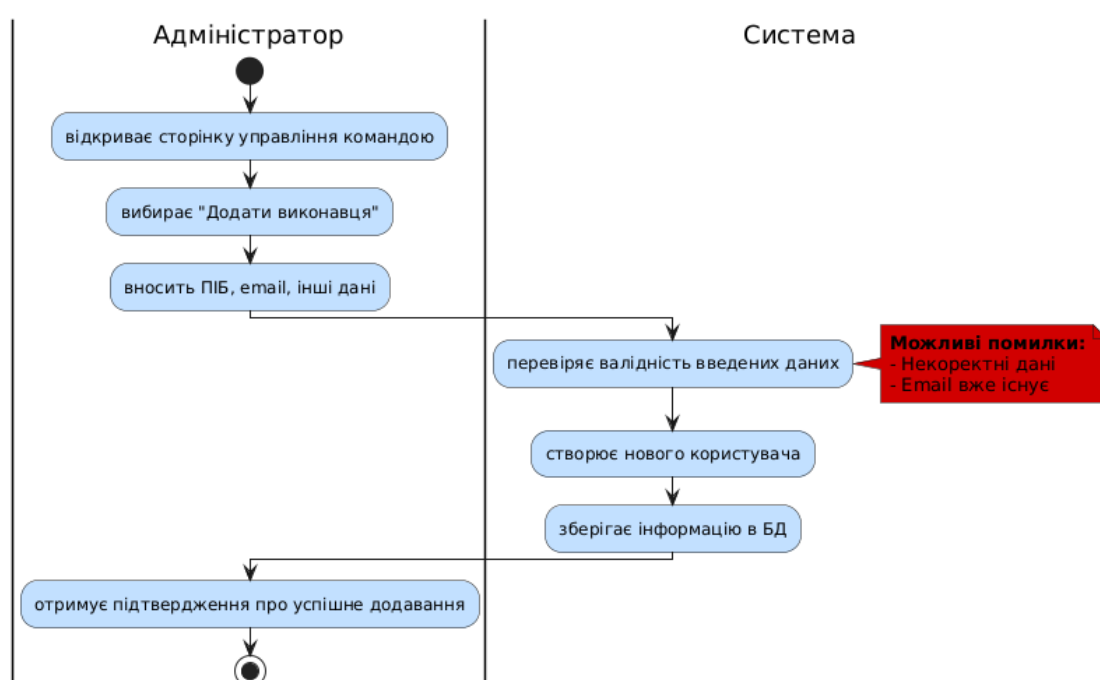


Рисунок 2.1 – Сценарій 1. Додавання нового виконавця

Адміністратор переходить до розділу "Завдання" і натискає кнопку для створення нового завдання. Після заповнення необхідних полів (назва, опис, дедлайн, виконавець) система перевіряє введені дані. Якщо дані валідні, система створює нове завдання в базі та прив'язує його до обраного виконавця. У разі помилок (порожні поля або некоректна дата) адміністратор отримує повідомлення про проблему. Після успішного створення адміністратор бачить підтвердження.



Рисунок 2.2 – Сценарій 2. Створення нового завдання

Адміністратор зі списку завдань обирає потрібне і натискає кнопку призначення виконавця. Він може вибрати одного або кількох виконавців для цього завдання. Система перевіряє права доступу адміністратора до редагування задачі і оновлює відповідні зв'язки у базі даних. Після цього адміністратор отримує підтвердження про успішне призначення.

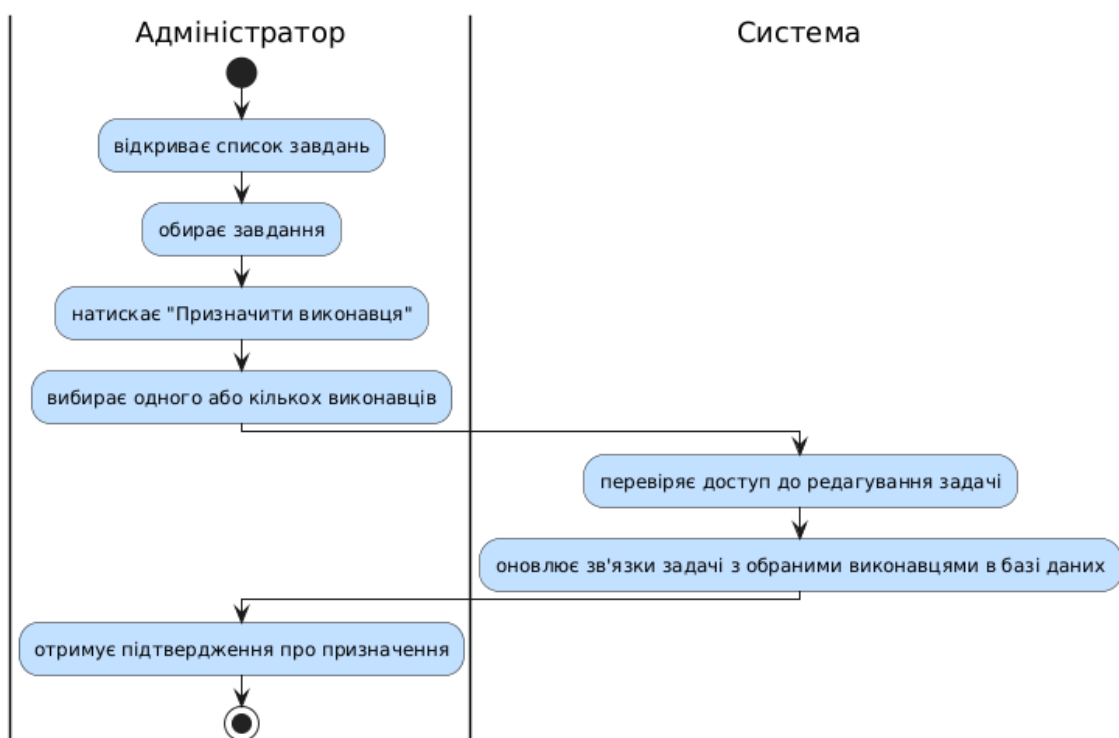


Рисунок 2.3 – Сценарій 3. Призначення завдання виконавцю

Виконавець переглядає список своїх завдань, обирає потрібне і змінює його статус на "Виконано". Система перевіряє права доступу, оновлює статус у базі даних і фіксує час змін. Після цього виконавець бачить оновлений статус.

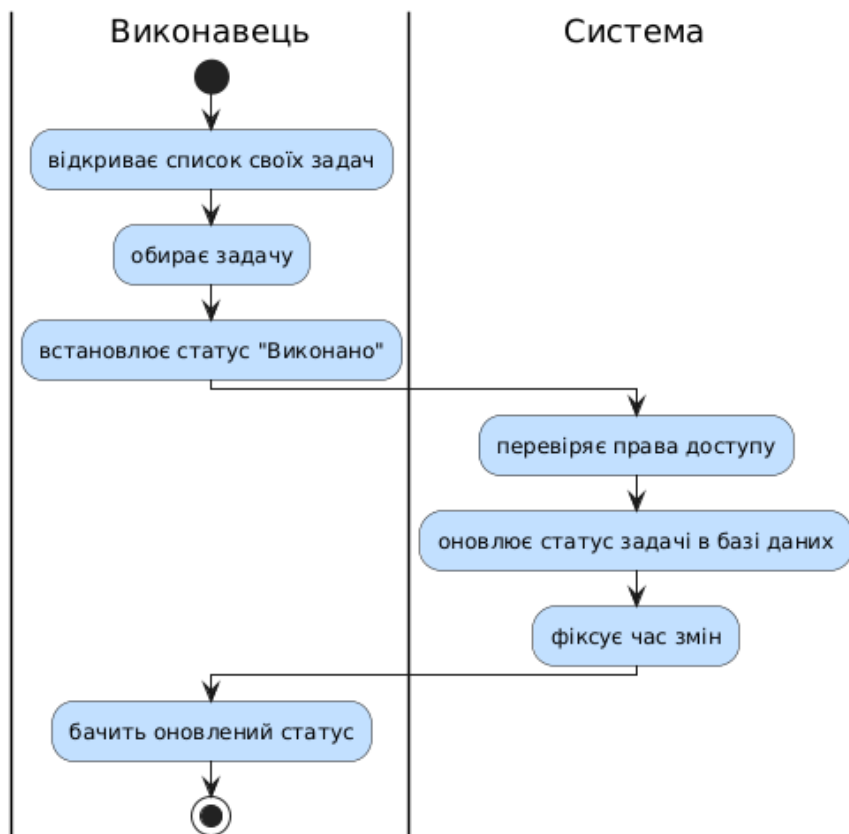


Рисунок 2.4 – Сценарій 4. Виконавець оновлює статус завдання

Адміністратор відкриває дашборд проєкту, де може переглянути загальну статистику по завданнях. Система автоматично розраховує кількість виконаних, невиконаних, прострочених завдань, підраховує відсоток виконання, аналізує навантаження на кожного виконавця і повертає зведену інформацію у вигляді графіків чи таблиць. Адміністратор на основі цієї інформації аналізує стан проєкту.

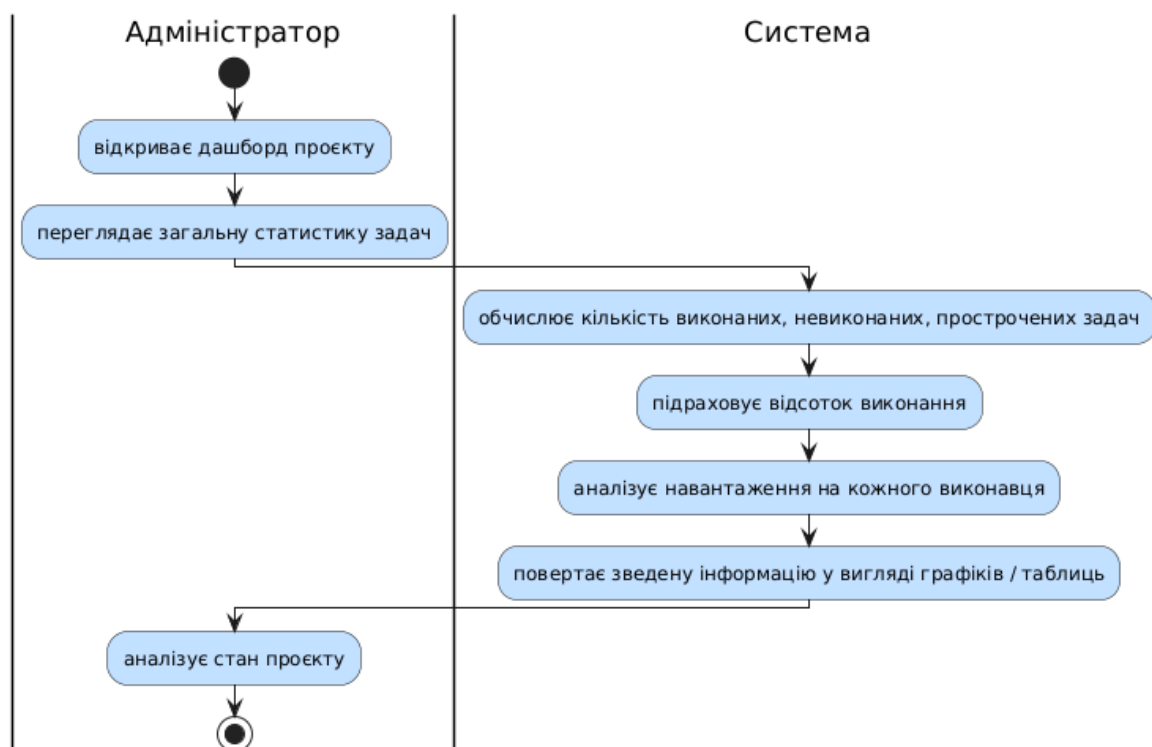


Рисунок 2.5 – Сценарій 5. Перевірка загального стану проєкту

Користувач вводить запит у поле пошуку, вибирає потрібні фільтри (статус, дедлайн, виконавець тощо) та запускає пошук. Система обробляє запит, фільтрує дані у базі завдань та виконавців за вибраними параметрами і повертає відповідний список результатів.

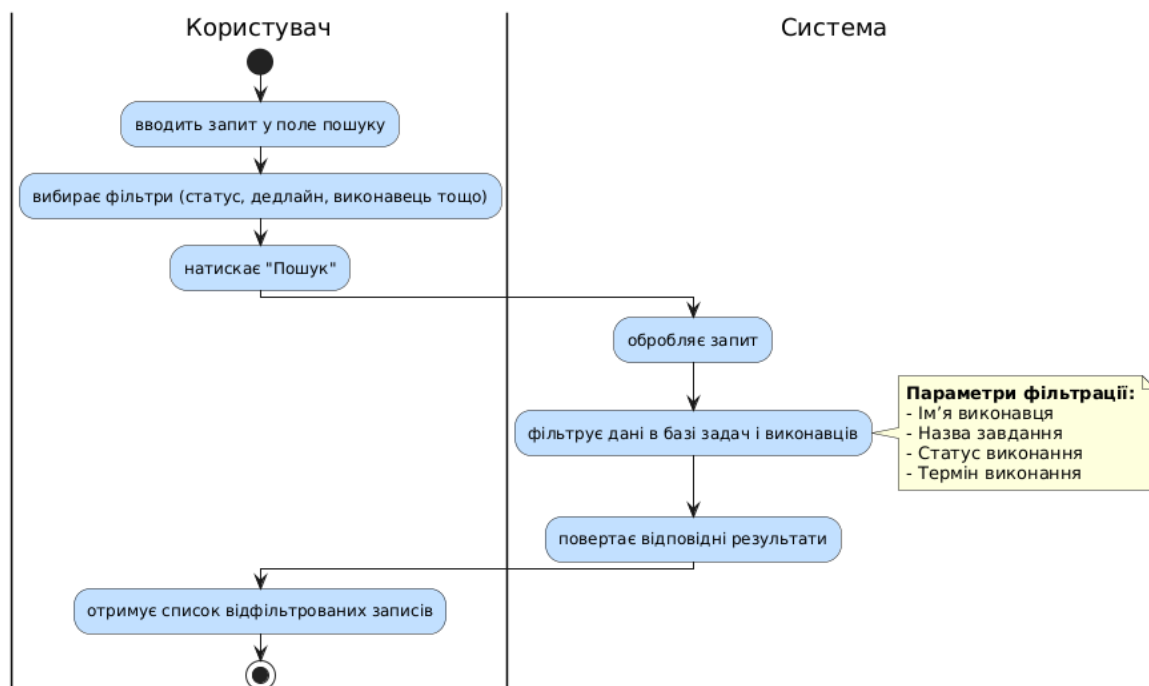


Рисунок 2.6 – Сценарій 6. Пошук задач або виконавців

2.2 Архітектура бази даних

Одним із ключових етапів проектування програмного забезпечення є побудова структури бази даних, яка визначає спосіб зберігання, організації та взаємозв'язку даних у системі. Оскільки розроблюваний застосунок оперує великою кількістю сутностей, що взаємодіють між собою, база даних повинна забезпечувати логічну цілісність, гнучкість при масштабуванні та підтримку складних зв'язків.

Для реалізації цих вимог у даному проекті було обрано реляційну базу даних PostgreSQL, яка забезпечує підтримку транзакцій, зовнішніх ключів, складних фільтрацій і гарантує стабільну роботу зі структурованими даними. Архітектура бази даних спроектована з урахуванням нормалізації, що дозволяє зменшити надлишковість даних та покращити їх цілісність.

2.2.1 Основні таблиці та їх структура

У розробленому застосунку для зберігання даних використовується реляційна база даних PostgreSQL. Структура бази даних проєктована з урахуванням нормалізації та майбутньої масштабованості системи.

Усього в базі даних передбачено чотири основні таблиці:

Таблиця 2.1 – users (Користувачі)

Назва поля	Тип даних	Опис
<i>id</i>	UUID	Первинний ключ
<i>full_name</i>	VARCHAR	ПІБ користувача
<i>email</i>	VARCHAR	Електронна пошта
<i>role</i>	VARCHAR	Роль у системі (admin / executor)
<i>created_at</i>	TIMESTAMP	Дата створення

Використовується для зберігання інформації про членів команди.

Таблиця 2.2 – projects (Проекти)

Назва поля	Тип даних	Опис
<i>id</i>	UUID	Первинний ключ
<i>name</i>	VARCHAR	Назва проєкту
<i>description</i>	TEXT	Опис
<i>created_by_id</i>	UUID	id користувача-менеджера
<i>created_at</i>	TIMESTAMP	Дата створення

Містить загальну інформацію про проєкти, в рамках яких відбувається розподіл завдань.

Таблиця 2.3 – tasks (Завдання)

Назва поля	Тип даних	Опис
<i>id</i>	UUID	Первинний ключ
<i>name</i>	VARCHAR	Назва завдання
<i>description</i>	TEXT	Деталі завдання

<i>project_id</i>	UUID	Зовнішній ключ на <code>projects</code>
<i>status</i>	VARCHAR	Статус (<code>todo</code> , <code>in_progress</code> , <code>done</code>)
<i>deadline</i>	DATE	Термін виконання
<i>progress</i>	INTEGER	Відсоток виконання
<i>created_at</i>	TIMESTAMP	Дата створення

Відповідає за зберігання інформації про завдання, що розподіляються між виконавцями.

Таблиця 2.4 – `task_assignments` (Призначення завдань)

Назва поля	Тип даних	Опис
<i>task_id</i>	UUID	Зовнішній ключ на <code>tasks</code>
<i>user_id</i>	UUID	Зовнішній ключ на <code>users</code>
<i>assigned_at</i>	TIMESTAMP	Дата призначення

Допоміжна таблиця для зберігання зв'язку "багато до багатьох" між задачами та виконавцями.

2.2.2 Схема зв'язків між таблицями

Для наочного подання структури бази даних застосовується ER-діаграма (Entity-Relationship Diagram), яка демонструє основні сутності системи, їх атрибути та характер зв'язків між таблицями. Такий підхід дозволяє чітко уявити логіку побудови даних та взаємодію між окремими компонентами інформаційної моделі.

У розробленому застосунку передбачено два основні типи зв'язків між таблицями — один до багатьох та багато до багатьох.

Зв'язок типу "один до багатьох" реалізується у випадках, коли одна сутність може мати декілька підлеглих. Наприклад, один користувач (ідентифікатор `users.id`) може створити кілька проєктів, де поле `created_by` у таблиці `projects` вказує на автора. Також один проєкт (поле `projects.id`) може містити багато завдань, про що свідчить наявність зовнішнього ключа `project_id` у таблиці `tasks`.

Зв'язок типу "багато до багатьох" реалізовано для відображення відношення між задачами та виконавцями. У даному випадку одна задача може бути призначена кільком користувачам, і водночас один виконавець може бути залучений до виконання багатьох задач. Така взаємодія реалізована через допоміжну таблицю `task_assignments`, яка містить пари ідентифікаторів задач і виконавців (`task_id` і `user_id` відповідно), тим самим фіксуючи призначення.

Запропонована схема зв'язків дозволяє гнучко масштабувати систему, розширювати її функціональність та забезпечує цілісність і взаємозв'язок даних.

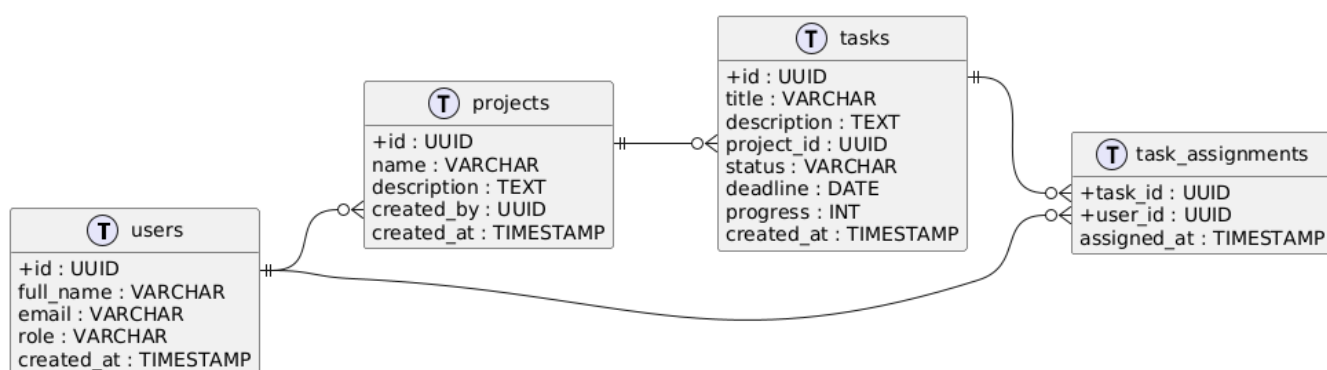


Рисунок 2.7 – ER-модель бази даних застосунку

2.3 Вибір технологій розробки

Під час реалізації програмного забезпечення було прийнято рішення використовувати стек технологій, який забезпечує швидку розробку, високу продуктивність, надійність та простоту підтримки. Вибір кожної технології обумовлений її відповідністю архітектурі застосунку, принципам модульності та масштабованості.

TypeScript був обраний як основна мова розробки як на клієнтській, так і на серверній частині. Він забезпечує статичну типізацію, що дозволяє виявляти помилки ще на етапі компіляції. Завдяки інтеграції з сучасними інструментами розробки (VS Code, ESLint, Vite), TypeScript підвищує якість коду та полегшує його супровід.

React використовується для побудови інтерфейсу користувача (частина View у шаблоні MVC). Ця бібліотека дозволяє розробляти інтерфейси у вигляді повторно використовуваних компонентів. React має велику спільноту, велику кількість бібліотек для розширення функціональності, а також забезпечує високу продуктивність завдяки віртуальному DOM.

Для реалізації серверної частини обрано Node.js з фреймворком Express. Node.js забезпечує неблокуючу модель обробки запитів, що особливо важливо для застосунків з великою кількістю одночасних користувачів. Express — це легкий і гнучкий фреймворк, який дозволяє швидко реалізувати REST API та ефективно структурувати серверну логіку. У загальній архітектурі MVC Express відповідає за контролери.

В якості системи керування базами даних обрано PostgreSQL — потужну реляційну БД з відкритим кодом. PostgreSQL підтримує складні зв'язки між таблицями, зовнішні ключі, транзакції, а також надає високий рівень безпеки та масштабованості.

Для взаємодії з базою даних використовується Prisma — сучасна ORM, яка інтегрується з TypeScript і дозволяє працювати з БД через типізовані об'єкти. Prisma забезпечує зручну генерацію моделей, автоматизацію міграцій, підтримку зв'язків між таблицями та автопідказки в редакторі коду. Крім того, Prisma суттєво пришвидшує розробку та підвищує надійність серверної частини.

Обраний стек технологій дозволяє ефективно реалізувати архітектуру MVC, забезпечити стабільність, масштабованість та зручність подальшої розробки і підтримки програмного забезпечення.

2.4 Використання шаблонів проєктування

З метою забезпечення гнучкої та масштабованої архітектури програмного забезпечення, на етапі проєктування системи передбачається використання декількох структурних і поведінкових шаблонів проєктування (патернів). Застосування

шаблонів дає змогу мінімізувати залежності між модулями, забезпечити розширюваність функціоналу та спростити супровід коду.

Один із ключових шаблонів, що планується до реалізації, — це Repository. Він буде використаний для ізоляції бізнес-логіки від специфіки збереження даних. Наприклад, для взаємодії з таблицями `users`, `tasks`, `projects` передбачено створення окремих репозиторіїв, які реалізовуватимуть типові операції читання, створення, оновлення та видалення даних. Це дозволить легко замінити джерело даних (наприклад, з PostgreSQL на іншу СКБД або мокові дані) без змін у бізнес-логіці.

Для реалізації гнучкої системи фільтрації та пошуку задач планується використання шаблону Strategy. У даному випадку буде створено набір окремих стратегій для фільтрації задач за статусом, терміном виконання, виконавцем тощо. Це дозволить змінювати або додавати фільтри без зміни існуючого коду, що відповідає принципу відкритості/закритості (OCP) у SOLID.

Шаблон Factory передбачається для створення об'єктів задач з різними параметрами — наприклад, з початковими статусами, дедлайнами, прив'язками до виконавців або проєктів. Використання фабрики дозволить централізовано контролювати процес створення задач та уникнути дублювання коду.

У разі реалізації сповіщень або реакції на певні дії (наприклад, зміна статусу задачі, оновлення дедлайну) можливе використання шаблону Observer або системи подій. Такий підхід забезпечить слабке зв'язування між об'єктами, які ініціюють події, та тими, що на них реагують.

Використання зазначених шаблонів дасть змогу забезпечити чисту, модульну та гнучку архітектуру, придатну для подальшого розширення функціональності без порушення існуючої логіки.

2.5 Застосування принципів ООП і SOLID

Архітектура застосунку спроектована з урахуванням основних принципів об'єктно-орієнтованого програмування (ООП), що дозволяє забезпечити модульність, повторне використання коду та зрозумілу структуру системи. Основними принципами ООП, що застосовуються у проєкті, є: інкапсуляція, успадкування, поліморфізм і абстракція.

Інкапсуляція реалізується через поділ логіки на окремі сервіси, класи та компоненти, кожен з яких має чітко визначені межі відповідальності. Абстракція проявляється у створенні інтерфейсів для роботи з різними сутностями (наприклад, `ITaskRepository`, `IUserService`), що дозволяє приховати реалізаційні деталі. Поліморфізм забезпечується можливістю створення узагальнених методів і функцій, що можуть працювати з різними об'єктами через спільні інтерфейси. Успадкування, хоча і використовується обмежено (на користь композиції), дозволяє розширювати функціональність базових класів, зберігаючи спільну логіку.

У межах архітектурного підходу також дотримано принципів SOLID, які є фундаментом якісного проєктування програмних систем:

- **Single Responsibility Principle (SRP)** – кожен клас або сервіс у системі виконує лише одну логічну функцію. Наприклад, `UserService` відповідає тільки за обробку користувачів, а `TaskService` — лише за логіку задач.
- **Open/Closed Principle (OCP)** – система дозволяє додавати нову поведінку (наприклад, нові типи фільтрації задач або статусів) без модифікації існуючих класів, що реалізовано через шаблон `Strategy`.
- **Liskov Substitution Principle (LSP)** – усі реалізації інтерфейсів повинні коректно працювати в тих місцях, де використовується базовий тип. Наприклад, якщо створюється кілька типів фільтрацій задач, вони мають бути взаємозамінними.

- **Interface Segregation Principle (ISP)** – інтерфейси, які реалізуються, не містять зайвої функціональності. Кожен інтерфейс проєктується таким чином, щоб реалізовуватись класом лише частково, відповідно до його логіки.
- **Dependency Inversion Principle (DIP)** – залежності реалізовані через абстракції, а не через конкретні реалізації. Наприклад, сервіс задач залежить від інтерфейсу репозиторію, а не від конкретної реалізації бази даних, що дозволяє легко проводити тестування, замінювати джерело даних або реалізовувати мок-версії.

Дотримання принципів ООП і SOLID сприяє створенню чистого, розширюваного та зрозумілого коду, що є критично важливим для систем з потенційно складною логікою взаємодії між компонентами.

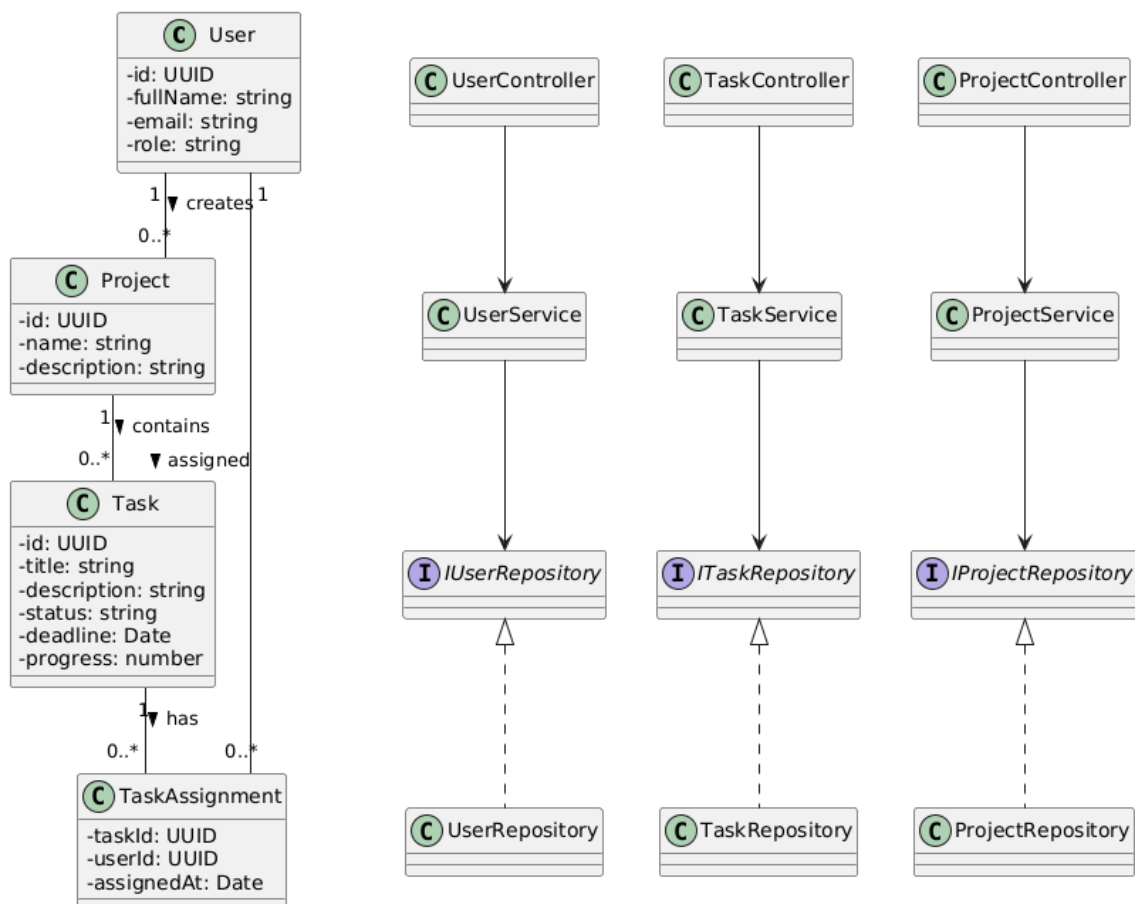


Рисунок 2.8 – UML-діаграма класів програмного забезпечення

РОЗДІЛ 3. РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1. Загальна структура коду

Програмне забезпечення розроблено за принципами архітектури MVC та поділено на дві логічні частини: клієнтську (фронтенд) та серверну (бекенд). Для зручності розробки кожна з частин розміщена в окремому репозиторії GitHub.

3.1.1 Frontend частина

Клієнтська частина реалізована з використанням React, TypeScript та Vite. Основна структура фронтенду:

```
task-planner-frontend/
├── src/
│   ├── api/
│   ├── components/
│   ├── hooks/
│   ├── lib/
│   ├── pages/
│   ├── styles/
│   ├── App.tsx
│   ├── Router.tsx/
│   └── main.tsx
├── public/
├── index.html
├── .env
├── vite.config.ts
└── package.json
```

Опис основних каталогів і файлів:

- `components/` – містить багаторазові React-компоненти, такі як списки завдань, форми тощо.
- `lib/` – зберігає типи, утиліти та загальні допоміжні модулі.
- `pages/` – компоненти сторінок застосунку (наприклад, панель управління).
- `services/` – реалізація логіки звернення до API бекенду (наприклад, запити на створення чи оновлення завдань).
- `styles/` – стилі, реалізовані через CSS Modules.
- `App.tsx` – кореневий компонент, що обгортає інтерфейс користувача і рендерить маршрути.
- `Router.tsx` – файл конфігурації маршрутизації між сторінками з використанням `createBrowserRouter()`.
- `main.tsx` – точка входу до застосунку.

3.1.2 Backend частина

Серверна частина побудована на Express.js з використанням TypeScript, Prisma ORM та PostgreSQL. Основна структура бекенду:

```
task-planner-backend/
├── prisma/
│   ├── migrations/
│   ├── schema.prisma
│   └── seed.ts
├── src/
│   ├── config/
│   ├── controllers/
│   ├── generated/
│   ├── middleware/
│   └── routes/
```

```

|   |— services/
|   |— tests/
|   |— utils/
|   |— validators/
|   |— server.ts
|   |— .env
|   |— package.json

```

Опис основних каталогів і файлів:

- `prisma/` – містить файл `schema.prisma` з описом моделей бази даних, міграції (`migrations/`) і файл `seed.ts` для початкового заповнення бази даних тестовими даними.
- `src/config/` – налаштування змінних середовища, параметри підключення до бази даних тощо.
- `src/controllers/` – обробники HTTP-запитів; приймають запити та викликають відповідні сервіси.
- `src/generated/` – автоматично згенеровані Prisma-клієнти для роботи з базою даних.
- `src/middleware/` – проміжні обробники запитів (наприклад, логування, обробка помилок, авторизація).
- `src/routes/` – визначення маршрутів API та їх зв'язок із контролерами.
- `src/services/` – бізнес-логіка, яка обробляє дані, викликає моделі та виконує розрахунки.
- `src/tests/` – тести для перевірки окремих модулів застосунку.
- `src/utils/` – допоміжні функції, які використовуються в різних частинах програми.
- `src/validators/` – логіка валідації вхідних даних, зокрема через `zod` чи подібні інструменти.
- `server.ts` – основний файл запуску серверу; ініціалізує додаток Express, підключає маршрути та middleware.

3.2. Реалізація моделі

У курсовому проєкті для взаємодії з базою даних використовується ORM Prisma, що забезпечує типізований та зручний доступ до PostgreSQL. Опис моделей міститься у файлі `prisma/schema.prisma`. Усі ідентифікатори реалізовано як `uuid`, а також визначено `enum`-типи для ролей користувачів і статусів завдань.

Модель `User` зберігає дані про користувачів:

```
model User {
  id          String          @id @default(uuid())
  fullName    String          @map("full_name")
  email       String          @unique
  password    String
  role        Role
  createdAt   DateTime        @default(now()) @map("created_at")
  projects    Project[]       @relation("CreatedProjects")
  assignments TaskAssignment[]
}
```

Кожен користувач має роль, адресу електронної пошти, дату реєстрації, а також пов'язаний з проєктами та завданнями, до яких його призначено.

Модель `Task` описує завдання в межах певного проєкту:

```
model Task {
  id          String          @id @default(uuid())
  name        String
  description  String?
  projectId   String          @map("project_id")
  status      TaskStatus
  deadline    DateTime?
  progress    Int             @default(0)
  createdAt   DateTime        @default(now()) @map("created_at")

  project     Project          @relation(fields: [projectId], references: [id])
  assignments TaskAssignment[]
}
```

Завдання мають назву, опис, дедлайн, статус `TaskStatus` та пов'язуються з проєктом і призначенням користувачів.

Інші моделі:

- `Project` — зберігає інформацію про проекти, які створюють користувачі.
- `TaskAssignment` — реалізує зв'язок багато-до-багатьох між користувачами та завданнями, фіксуючи час призначення.

Перерахування (enum):

- `Role: ADMIN, MANAGER, USER`
- `TaskStatus: UNASSIGNED, ASSIGNED, IN_PROGRESS, PENDING_REVIEW, COMPLETED`

Ці enum-типи дозволяють централізовано керувати правами доступу та етапами виконання завдань.

3.3. Реалізація контролерів

У проєкті контролери відповідають за обробку HTTP-запитів, які надходять з клієнтської частини (React), і взаємодіють з відповідними сервісами, що реалізують бізнес-логіку. Вони розміщені в каталозі `src/controllers/`.

Усі контролери реалізовано як окремі функції, що експортуються з відповідних файлів. Наприклад, у `task.controller.ts` описано методи для створення, отримання, оновлення та видалення завдань.

Приклад: `getAllTasks` з `task.controller.ts`:

```
import { Request, Response } from 'express';
import * as taskService from '@services/task.service';

export const getAllTasks = async (_req: Request, res: Response) => {
  try {
    const tasks = await taskService.getAllTasks();
    res.json(tasks);
  } catch (error) {
    res.status(500).json({ error: 'Failed to get tasks', details: error });
  }
};
```

Цей контролер обробляє запит `GET /tasks` та повертає список усіх завдань, отриманих із сервісу `taskService`.

Інші контролери реалізовано аналогічно:

- `user.controller.ts` – створення та отримання користувачів.
- `project.controller.ts` – логіка роботи з проєктами.
- `taskAssignment.controller.ts` – призначення користувачів до завдань.

Контролери відповідають лише за прийом запиту, обробку помилок і повернення відповіді. Уся логіка винесена у відповідні сервіси, що відповідає принципам розділення обов'язків.

3.4. Реалізація представлення (View)

Інтерфейс користувача реалізовано з використанням `React + TypeScript`. Основна увага приділялась створенню зрозумілої, функціональної та адаптивної структури з акцентом на повторне використання компонентів.

Одним із найскладніших і водночас найцікавіших елементів реалізації став контекст для роботи з проєктами, що дозволяє централізовано отримувати стан `projects` у будь-якому компоненті.

Використання контексту `useProjects()`:

У проєкті реалізовано контекст `ProjectContext`, який забезпечує доступ до масиву проєктів, статусу завантаження та можливих помилок. Усі дані завантажуються у провайдері `ProjectProvider`:

```
// ProjectsProvider.tsx
<ProjectContext.Provider value={{ projects, loading, error }}>
  {children}
</ProjectContext.Provider>
```

А вже в будь-якому компоненті їх можна отримати за допомогою кастомного хука `useProjects()`:

```
// HomePage.tsx
const { projects, loading, error } = useProjects();
```

Це дозволяє уніфікувати логіку роботи з даними та значно спрощує структуру компонента.

Рендеринг `ProjectsList` через пропси:

На головній сторінці (`HomePage.tsx`) проєкти передаються в компонент `ProjectsList` як пропс:

```
const HomePage: React.FC = () => {
  const { projects, loading, error } = useProjects();

  if (loading) return <StatusMessage variant="loading">Завантаження
проектів...</StatusMessage>;
  if (error) return <StatusMessage variant="error">{error}</StatusMessage>;

  return <ProjectsList projects={projects} />;
};
```

Цей підхід дозволяє зберігати компонент `ProjectsList` максимально чистим і універсальним, оскільки він не залежить від способу отримання даних, а лише від отриманого `projects[]`.

Такий підхід звільняє від проблеми "prop drilling" — це коли дані (наприклад, список проєктів) передаються через кілька рівнів компонентів, навіть якщо проміжні компоненти їх не використовують.

Використовуючи контекст + пропси, ми:

- централізуємо доступ до даних через `ProjectProvider`;
- виводимо конкретні компоненти з контексту — залишаючи їм лише чисту відповідальність за відображення;
- забезпечуємо розширюваність і тестованість системи.

Загально кажучи про інтерфейс користувача, візуально інтерфейс витримано в мінімалістичному, сучасному стилі, з чітким розділенням блоків, адаптивними картками проєктів і повідомленнями про статус (завантаження, помилки, відсутність проєктів).

Це забезпечує зрозумілу взаємодію користувача з системою і створює приємний досвід роботи з вебзастосунком.

3.5. Логіка взаємодії між компонентами

У системі реалізовано взаємодію між компонентами за архітектурним шаблоном MVC (Model–View–Controller). Застосунок поділено на окремі частини — інтерфейс користувача (View), серверну логіку (Controller) і моделі даних (Model). Такий підхід забезпечує гнучкість, масштабованість та зрозумілу структуру взаємодії між модулями.

Послідовність взаємодії:

1. Користувач взаємодіє з UI-компонентами, реалізованими у React (наприклад, заповнює форму або відкриває сторінку).
2. Компоненти використовують `axios`, щоб надіслати HTTP-запит до API бекенду (Express.js).
3. Контролер (Controller) обробляє запит, перевіряє дані, викликає відповідний сервіс (Service).
4. Сервіс звертається до бази даних через Prisma (Model) та повертає оброблену відповідь.
5. Контролер відправляє відповідь назад на фронтенд.
6. View-компоненти оновлюють інтерфейс відповідно до отриманих даних.

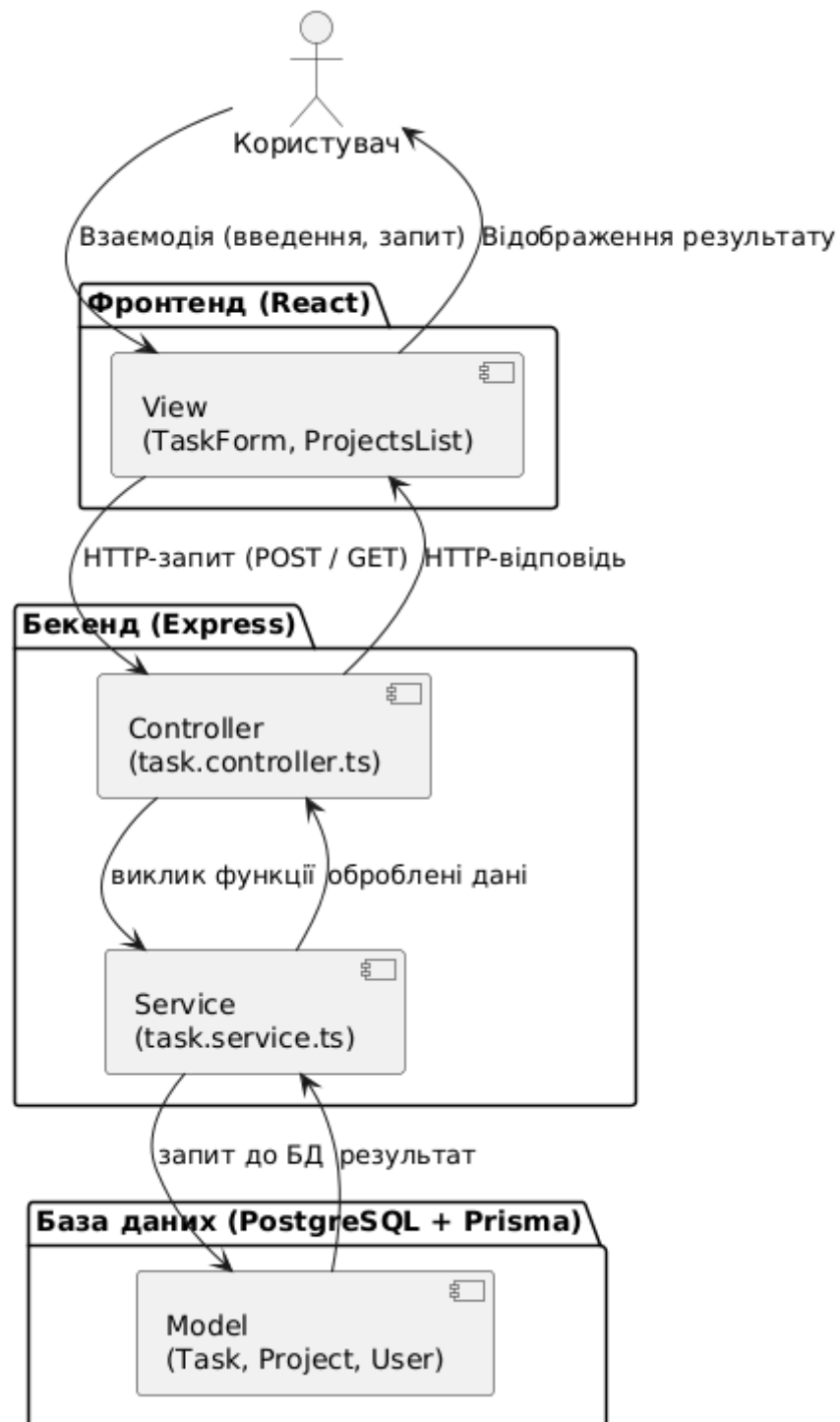


Рисунок 3.1 – Схема взаємодії компонентів шаблону MVC у проєкті

Приклад у коді:

- **Фронт (React):** компонент `TaskForm` надсилає запит на `/api/tasks`.
- **Бекенд (Express):** `task.controller.ts` приймає запит і викликає `createTask()` із `task.service.ts`.

- `task.service.ts` взаємодіє з `prisma.task.create`, створює запис у базі.
- Дані повертаються через JSON і відображаються у списку `TaskList`.

Ця структура забезпечує чіткий поділ відповідальностей, дозволяє легко тестувати і масштабувати кожен шар окремо, а також зберігає чисту архітектуру проєкту навіть із додаванням нових функцій.

3.6. Репозиторій з кодом проєкту

Для забезпечення зручного доступу до вихідного коду курсового проєкту було створено публічні репозиторії на GitHub, які містять реалізацію обох частин системи — клієнтської та серверної. Кожен репозиторій має структуру, описану у пункті 3.1, та включає весь необхідний код, конфігураційні файли, приклади запитів і типізацію.

- Фронтенд (React + TypeScript + Vite)
- Бекенд (Express + TypeScript + Prisma + PostgreSQL)

За посиланнями на репозиторії див. [Додаток А](#)

РОЗДІЛ 4. ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1. Підходи до тестування в архітектурі MVC

У даному проєкті було реалізовано архітектуру типу Model–View–Controller (MVC), яка дозволяє чітко розмежувати відповідальність між компонентами системи: представленням (View), логікою (Controller), бізнес-логікою (Service) та моделлю даних (Model). Це дало змогу зосередити тестування саме на сервісному рівні, де відбувається обробка основних операцій із даними.

Тестування проводилося шляхом написання модульних тестів для кожного окремого сервісного модуля. Це забезпечує перевірку правильності реалізації бізнес-логіки без необхідності підключення до реальної бази даних. Для цього було використано бібліотеку Vitest, яка дозволяє створювати ізольовані тести з використанням мок-об'єктів. Компонент Prisma (ORM) було змочано вручну, що дозволило емулювати взаємодію з базою даних без прямого з'єднання.

Такий підхід до тестування забезпечив швидке виконання тестів, зручність налагодження та високу надійність внутрішньої логіки сервісів.

4.2. Методи тестування

4.2.1. Модульне тестування

Модульне тестування є базовим типом тестування, що полягає у перевірці окремих функцій або методів у ізольованому середовищі. У межах курсової роботи основна увага приділялася саме цьому підходу, оскільки вся бізнес-логіка проєкту реалізована у сервісному рівні, відповідно до архітектури MVC.

Для реалізації тестів використовувалася бібліотека Vitest, яка забезпечує швидкий запуск, зручне мокання залежностей та інтеграцію з TypeScript. Об'єкт Prisma Client, через який здійснюється взаємодія з базою даних, був змочаний за

допомогою `vi.fn()`, що дозволило уникнути підключення до реальної БД під час тестування.

Було протестовано такі сервіси:

- `project.service.ts` — створення, оновлення, видалення проєкту, розрахунок статусу проєкту;
- `task.service.ts` — створення, оновлення, фільтрація та видалення завдань із врахуванням статусу;
- `user.service.ts` — CRUD-операції над користувачами, пошук за ім'ям або email;
- `taskAssignment.service.ts` — призначення та відв'язка користувачів від завдань, оновлення призначень.

Усього було написано понад 25 модульних тестів, а якщо бути точнішим – 37, які покривають основні гілки логіки. Нижче подано приклад одного з таких тестів:

```
describe("updateTaskStatus", () => {
  it("оновлює статус завдання", async () => {
    const result = await updateTaskStatus(1, "done");
    expect(result.status).toBe("done");
  });
});
```

Модульне тестування дозволило переконатися у правильності поведінки ключових функцій системи в умовах, наближених до реального використання (див. [Додаток Б](#)).

Результати модульного тестування після виконання команди `npm run test:coverage`, яка генерує звіт про покриття коду тестами, показали високий рівень перевірки сервісної логіки. У консолі відобразився текстовий звіт із відсотками покриття по кожному модулю, а також було автоматично згенеровано HTML-звіт (`coverage/index.html`), який наочно демонструє, які рядки коду були протестовані (див. [Додаток В](#)).

4.2.2. Інтеграційне тестування

Інтеграційне тестування передбачає перевірку взаємодії між різними компонентами системи — наприклад, контролерами, сервісами та базою даних. У межах цієї курсової роботи було вирішено зосередитися на модульному рівні, оскільки основна логіка проєкту винесена у сервіси, які вже були детально протестовані.

Для перевірки коректності роботи функцій використовувалося мокання ORM-клієнта Prisma. Такий підхід дозволив ізолювати бізнес-логіку від реального середовища зберігання даних і зосередитися саме на логіці обробки даних. Це також значно спростило процес тестування, підвищило швидкість запуску тестів і виключило залежність від стану зовнішньої бази даних.

Таким чином, інтеграційне тестування не проводилось, оскільки тестування сервісного рівня виявилось достатнім для забезпечення надійності та передбачуваної поведінки системи.

4.2.3. Функціональне тестування

Функціональне тестування проводилося вручну за допомогою програми Postman. Мета такого тестування — перевірити, чи коректно працює система на рівні HTTP-запитів: чи виконується обробка запитів, чи повертаються правильні статуси відповіді, чи оновлюються дані в базі тощо.

Під час тестування (див. [Додаток Г](#)) були перевірені основні маршрути API, що відповідають за створення, оновлення та видалення користувачів, задач і проєктів. Також тестувалась функція призначення користувача до завдання та зміна статусу задачі.

Це дозволило переконатися в правильності поведінки серверної частини в умовах, максимально наближених до реального використання системи кінцевими користувачами.

4.3. Аналіз результатів тестування

У результаті проведеного модульного тестування було перевірено основні функції чотирьох сервісів: `project.service.ts`, `task.service.ts`, `user.service.ts` та `taskAssignment.service.ts`. Усі тести виконувалися ізольовано, з використанням моків для Prisma-клієнта, що дозволило забезпечити стабільність і незалежність від зовнішніх джерел даних.

Після запуску команди `npm run test` усі модульні тести було пройдено успішно. Для перевірки глибини покриття коду тестами було використано команду `npm run test:coverage`, яка сформувала текстовий звіт у терміналі та HTML-звіт у файлі `coverage/index.html`. Зі звіту видно, що сервіс `project.service.ts` має 100% покриття коду, а інші сервіси також охоплюють більшість логіки, включаючи умовні гілки, функції та оператори.

Крім того, вручну було проведено функціональне тестування HTTP-запитів через Postman. Перевірялися маршрути створення, редагування та видалення задач, користувачів, проєктів, а також призначення виконавців і зміна статусу задач. Усі запити пройшли успішно, підтвердивши коректність поведінки API в типових сценаріях.

Отже, на основі результатів тестування можна зробити висновок, що система функціонує стабільно, бізнес-логіка реалізована коректно, а основні сервіси мають високу надійність. Тестове покриття є достатнім для виявлення помилок на рівні логіки, що значно підвищує якість та підтримуваність проєкту.

ВИСНОВКИ

У межах виконання курсової роботи було створено вебзастосунок «Планувальник завдань», який забезпечує ефективне управління робочими процесами в команді. Розробка системи здійснювалась із дотриманням архітектури MVC, що дозволило розділити логіку між рівнями представлення (frontend на React + TypeScript), обробки запитів (Express.js) та доступу до даних (Prisma ORM + PostgreSQL). Така структура забезпечила зручність масштабування, модульність коду та можливість подальшого розширення функціональності.

На етапі реалізації було створено повноцінний бекенд, який охоплює CRUD-операції з користувачами, проєктами, завданнями та призначеннями виконавців. Особлива увага приділялася якості коду та покриттю логіки тестами. Для перевірки коректності роботи сервісного рівня було реалізовано модульне тестування за допомогою бібліотеки Vitest, що дозволило ізолювати бізнес-логіку від зовнішніх залежностей. Усього написано 37 юніт-тестів, які охопили всі основні функції чотирьох сервісів — project, task, user та taskAssignment.

Результати запуску `npm run test:coverage` підтвердили, що весь код сервісів охоплений тестами на 100% — як за кількістю операторів, умовних гілок, функцій, так і за рядками. Це свідчить про високу якість реалізації, надійність логіки та впевненість у стабільності системи. Додатково було проведено функціональне тестування HTTP-запитів через Postman — протестовані операції створення користувачів, проєктів, задач, призначення виконавців та зміни статусів. Всі перевірки пройдено успішно, що підтверджено відповідними відповідями сервера.

Підсумовуючи, у ході роботи були повністю реалізовані всі заплановані етапи — від формування вимог до впровадження та тестування системи. Отриманий застосунок є стабільним, тестованим і зручним для використання. Він може бути застосований як у навчальних цілях, так і в реальних умовах для організації командної роботи на малих і середніх проєктах.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Мартін Р. Чистий код. Створення, аналіз та рефакторинг. – Київ: Діалектика, 2019. – 432 с.
2. Шлеє Г. React та TypeScript: Професійна розробка фронтенду. – К.: Наука і техніка, 2022. – 304 с.
3. Офіційна документація React. – <https://react.dev/> – (дата звернення: 05.05.2025).
4. Офіційна документація Express.js. – <https://expressjs.com/> – (дата звернення: 05.05.2025).
5. Prisma ORM Documentation. – <https://www.prisma.io/docs> – (дата звернення: 05.05.2025).
6. Vitest – Blazing Fast Unit Test Framework. – <https://vitest.dev> – (дата звернення: 05.05.2025).
7. Postman API Platform Documentation. – <https://www.postman.com> – (дата звернення: 05.05.2025).
8. TypeScript Handbook. – <https://www.typescriptlang.org/docs/> – (дата звернення: 05.05.2025).
9. Clean Architecture: A Craftsman's Guide to Software Structure and Design / Robert C. Martin. – Pearson Education, 2017. – 432 p.

ДОДАТКИ

Фронтенд частина проєкту

Посилання на репозиторій: <https://github.com/luckyscammer/task-planner-frontend>

Бекенд частина проєкту

Посилання на репозиторій: <https://github.com/luckyscammer/task-planner-backend>

Фрагменти модульних тестів

- Тестування сервісу `project.service.ts`:

```
describe('getProjectStatus', () => {
  it('повертає completed, якщо всі завдання виконані', async () => {
    prisma.task.count
      .mockResolvedValueOnce(5) // totalTasks
      .mockResolvedValueOnce(5) // completedTasks

    const result = await getProjectStatus('1')
    expect(result.progress).toBe(100)
    expect(result.state).toBe('completed')
  })
})
```

Фрагмент тесту, який перевіряє правильність розрахунку прогресу проєкту.

- Тестування сервісу `task.service.ts`:

```
it('створює задачу зі статусом COMPLETED і встановлює progress = 100', async () => {
  const input = { title: 'New Task', status: TaskStatus.COMPLETED }
  const result = await createTask(input as any)

  expect(result.progress).toBe(100)
})
```

Фрагмент тесту створення задачі зі статусом, що автоматично встановлює прогрес.

- Тестування сервісу `user.service.ts`:

```
it('шукає користувачів за email або ім'ям (insensitive)', async () => {
  const result = await searchUsers('test')
  expect(prisma.user.findMany).toHaveBeenCalledWith({
    where: {
      OR: [
        { fullName: { contains: 'test', mode: 'insensitive' } },
        { email: { contains: 'test', mode: 'insensitive' } },
      ],
    },
  })
})
```


Фрагмент тесту, який перевіряє пошук користувачів за частковим збігом.

– Тестування сервісу `taskAssignment.service.ts`:

```
it('видаляє призначення користувача до задачі за композитним ключем', async () => {  
  const result = await unassignUserFromTask('u1', 't1')  
  expect(prisma.taskAssignment.delete).toHaveBeenCalledWith({  
    where: { taskId_userId: { userId: 'u1', taskId: 't1' } },  
  })  
})
```

Фрагмент тесту, який перевіряє видалення призначення користувача із задачі.

Результати test:coverage

```
RUN v3.1.3 D:/task-planner-backend
Coverage enabled with v8

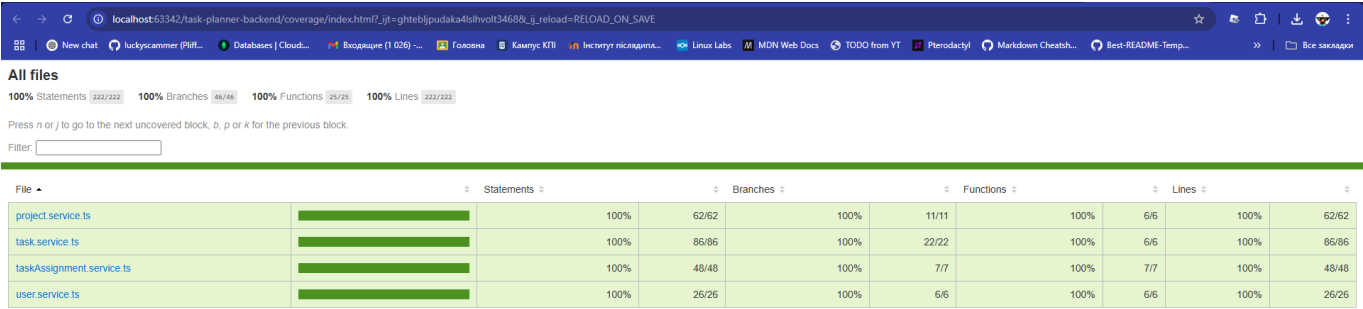
✓ tests/taskAssignment.service.test.ts (7 tests) 10ms
✓ tests/user.service.test.ts (6 tests) 12ms
✓ tests/project.service.test.ts (8 tests) 8ms
✓ tests/task.service.test.ts (16 tests) 13ms

Test Files 4 passed (4)
Tests 37 passed (37)
Start at 16:13:10
Duration 774ms (transform 262ms, setup 0ms, collect 528ms, tests 43ms, environment 1ms, prepare 753ms)

% Coverage report from v8
-----|-----|-----|-----|-----|-----
File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 100 | 100 | 100 | 100 |
project.service.ts | 100 | 100 | 100 | 100 |
task.service.ts | 100 | 100 | 100 | 100 |
taskAssignment.service.ts | 100 | 100 | 100 | 100 |
user.service.ts | 100 | 100 | 100 | 100 |
-----|-----|-----|-----|-----|-----
```

Рисунок В.1 – Текстовий звіт покриття коду після виконання команди `npm run test:coverage`

На зображенні показано успішне проходження 37 тестів, що охоплюють чотири сервісні модулі (project, task, user, taskAssignment). Всі файли мають 100% покриття по всіх метриках: оператори (statements), гілки (branches), функції (funcs) та рядки (lines).



File	Statements	Branches	Functions	Lines
project.service.ts	100%	100%	100%	100%
task.service.ts	100%	100%	100%	100%
taskAssignment.service.ts	100%	100%	100%	100%
user.service.ts	100%	100%	100%	100%

Рисунок В.2 – HTML-звіт покриття коду (coverage/index.html)

HTML-звіт дає візуальне представлення рівня покриття коду. Усі чотири сервісні файли мають по 100% виконаних інструкцій, перевірених умов і функцій, що свідчить про повне охоплення ключової логіки проєкту модульними тестами.

Фрагменти функціонального тестування через Postman

Розглядати запити будемо у форматі:

Таблиця Г.1 – Приклад формату запиту

Дія	Метод	Endpoint	Body-приклад	Очікувана Відповідь
-----	-------	----------	--------------	---------------------

Огляд запитів

Таблиця Г.2 – Структура запиту «Створення користувача»

Дія	Метод	Endpoint	Body-приклад	Очікувана Відповідь
Створення користувача	POST	/api/users	{ "fullName": "Test User", "email": "test@example.com", "password": "testuvanya", "role": "USER" }	Статус 201 + user id

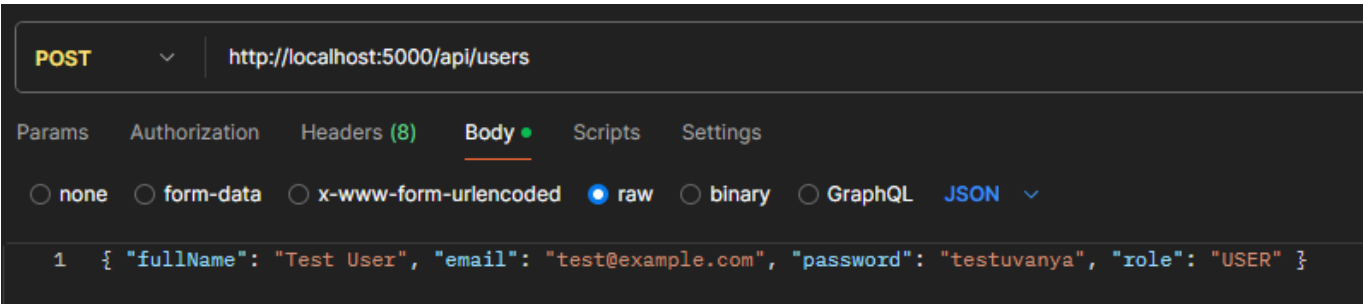


Рисунок Г.1 – Тіло запиту в Postman

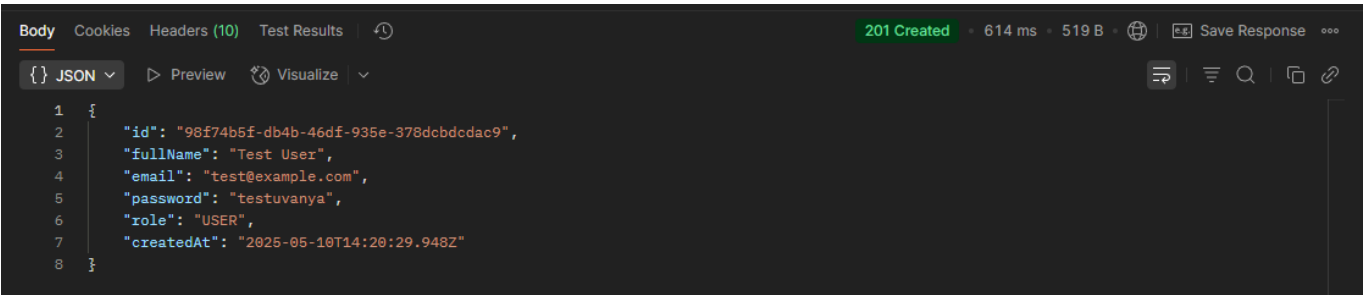


Рисунок Г.2 – Результат запиту в Postman

Таблиця Г.3 – Структура запиту «Створення проєкту»

Дія	Метод	Endpoint	Body-приклад	Очікувана Відповідь
Створення проєкту	POST	/api/projects	<pre>{ "name": "Test", "description": "Testing project", "createdById": "<user_id>" }</pre>	Статус 201 + project object

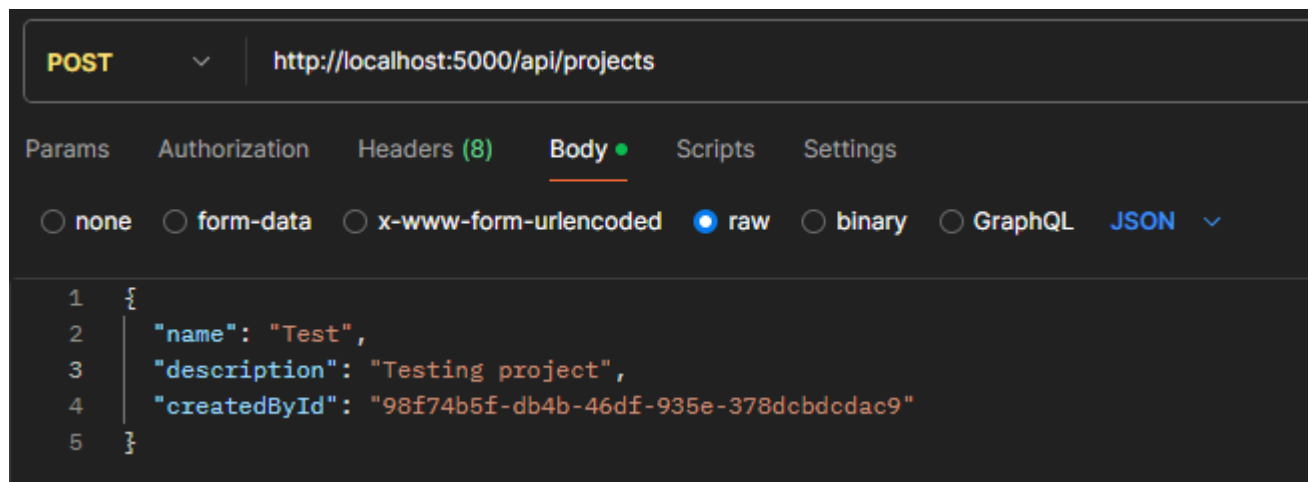


Рисунок Г.3 – Тіло запиту в Postman

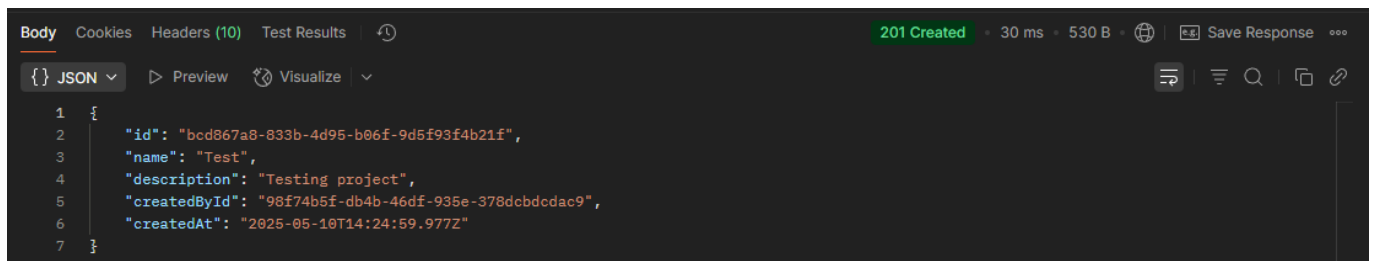


Рисунок Г.4 – Результат запиту в Postman

Таблиця Г.4 – Структура запиту «Створення задачі»

Дія	Метод	Endpoint	Body-приклад	Очікувана Відповідь
Створення задачі	POST	/api/tasks	{ "name": "Доробити бекенд", "status": "IN_PROGRESS", "projectId": "<project_id>" }	Статус 201 + task object

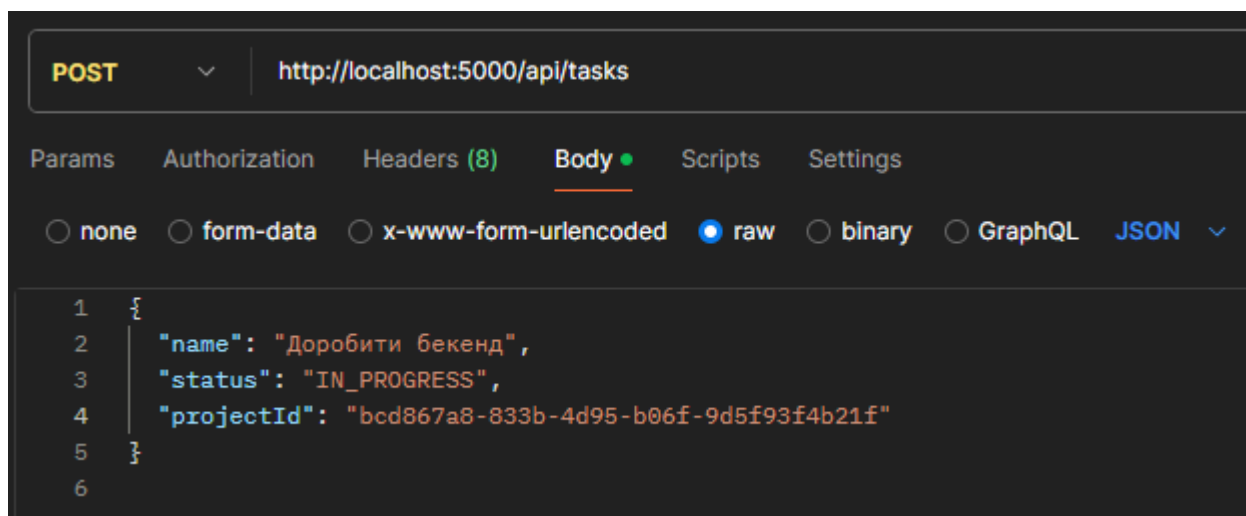


Рисунок Г.5 – Тіло запиту в Postman



Рисунок Г.6 – Результат запиту в Postman

Таблиця Г.5 – Структура запиту «Призначення користувача до задачі»

Дія	Метод	Endpoint	Body-приклад	Очікувана Відповідь
Призначення користувача до задачі	POST	/api/assignments	{ "userId": "<user_id>", "taskId": "<task_id>" }	Статус 201 + assignment

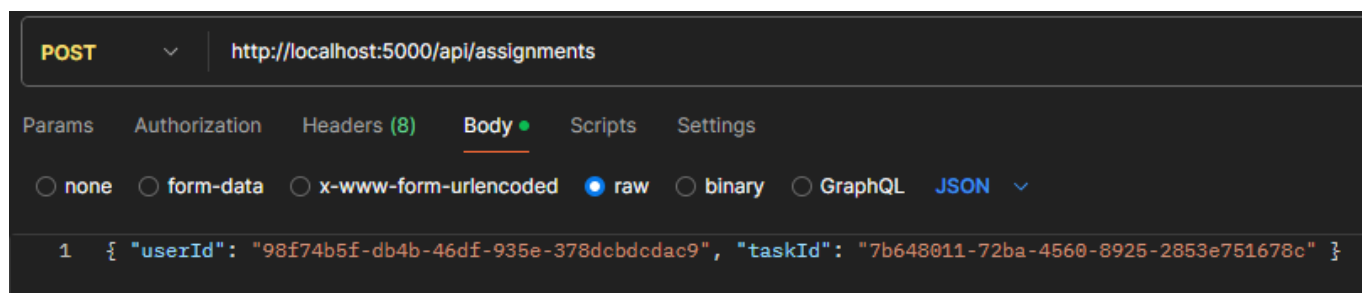


Рисунок Г.7 – Тіло запиту в Postman



Рисунок Г.8 – Результат запиту в Postman

Таблиця Г.6 – Структура запиту «Зміна статусу задачі»

Дія	Метод	Endpoint	Body-приклад	Очікувана Відповідь
Зміна статусу задачі	PUT	/api/tasks/<task_id>	{ "status": "COMPLETED" }	Статус 200 + task

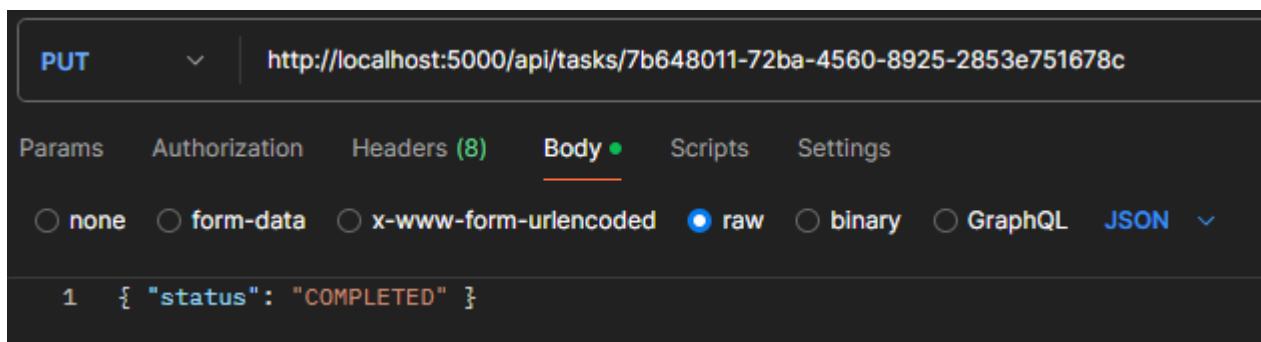


Рисунок Г.9 – Тіло запиту в Postman

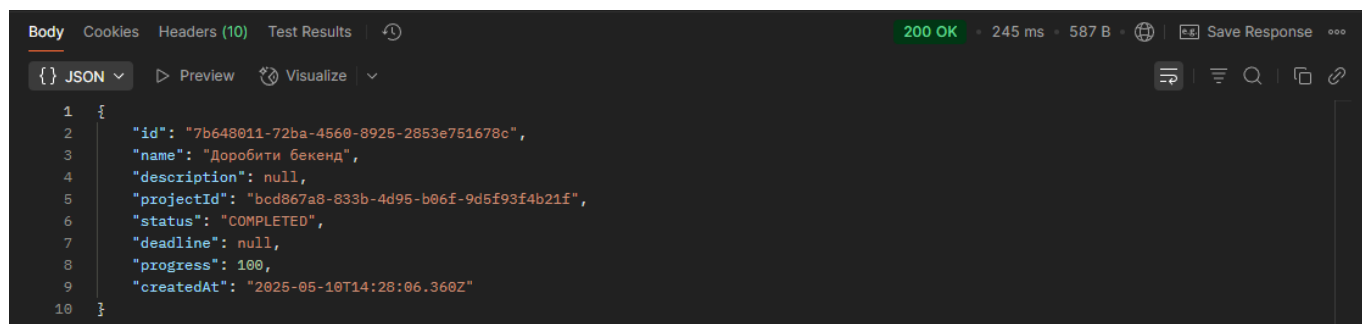


Рисунок Г.10 – Результат запиту в Postman